PROGRAM 10: Build a Document-Based AI Assistant Supporting Multiple Documents

AIM

To develop a document-based AI assistant that processes multiple uploaded text documents and answers questions using relevant information from the documents.

ALGORITHM

1. Install the required libraries.
2. Load an embedding model and a text-generation model.
3. Upload multiple text files using Google Colab.
4. Read the contents of the uploaded files.
5. Split the documents into smaller chunks.
6. Generate embeddings for all chunks.
7. Store the embeddings in a FAISS vector database.
8. Accept a question from the user.
9. Retrieve the most relevant chunks.
10. Generate and display an answer using the retrieved context.

CODE

Cell 1: Install libraries

```
!pip -q install sentence-transformers faiss-cpu transformers sentencepiece
```

Cell 2: Python program

```
import faiss

import numpy as np

import torch

from google.colab import files

from sentence_transformers import SentenceTransformer

from transformers import pipeline


# Upload multiple .txt files

print("Upload one or more TXT documents:")

uploaded = files.upload()
```

```
# Read uploaded files
documents = []

for filename, content in uploaded.items():
    if filename.lower().endswith(".txt"):
        text = content.decode("utf-8", errors="ignore")
        documents.append((filename, text))

if not documents:
    raise ValueError("Please upload at least one .txt file.")

# Split text into chunks
chunks = []
metadata = []

for filename, text in documents:
    words = text.split()
    chunk_size = 100
    overlap = 20

    for start in range(0, len(words), chunk_size - overlap):
        chunk = " ".join(words[start:start + chunk_size])

        if chunk.strip():
            chunks.append(chunk)
            metadata.append(filename)

if not chunks:
    raise ValueError("The uploaded files contain no usable text.")
```

```
# Load embedding model
embedder = SentenceTransformer("all-MiniLM-L6-v2")

# Generate chunk embeddings
vectors = embedder.encode(
    chunks,
    convert_to_numpy=True
).astype("float32")

faiss.normalize_L2(vectors)

# Build FAISS database
index = faiss.IndexFlatIP(vectors.shape[1])
index.add(vectors)

# Load generation model
generator = pipeline(
    "text2text-generation",
    model="google/flan-t5-small",
    device=0 if torch.cuda.is_available() else -1
)

print("\nDocuments indexed:", len(documents))
print("Text chunks indexed:", len(chunks))

# Question answering loop
while True:
    question = input("\nAsk a question (or type exit): ")

    if question.lower().strip() == "exit":
```

```
    print("Assistant: Goodbye!")
    break

q_vector = embedder.encode(
    [question],
    convert_to_numpy=True
).astype("float32")

faiss.normalize_L2(q_vector)

k = min(3, len(chunks))
scores, ids = index.search(q_vector, k)

context = "\n".join(
    f"Source: {metadata[i]}\n{chunks[i]}"
    for i in ids[0]
)

prompt = (
    "Answer the question using the supplied documents. "
    "If the answer is not available, say so.\n"
    f"Context: {context}\n"
    f"Question: {question}\n"
    "Answer:"
)

answer = generator(
    prompt,
    max_new_tokens=120,
    do_sample=False
```

```
    )[0]["generated_text"]

    print("\nRetrieved Sources:")
    for i in ids[0]:
        print("-", metadata[i])

    print("\nAssistant:", answer)
```

INPUT

Upload two or more .txt files containing your study material.

Example question:

What is artificial intelligence?

OUTPUT

Upload one or more TXT documents:

Documents indexed: 2

Text chunks indexed: [calculated number]

Ask a question (or type exit):

What is artificial intelligence?

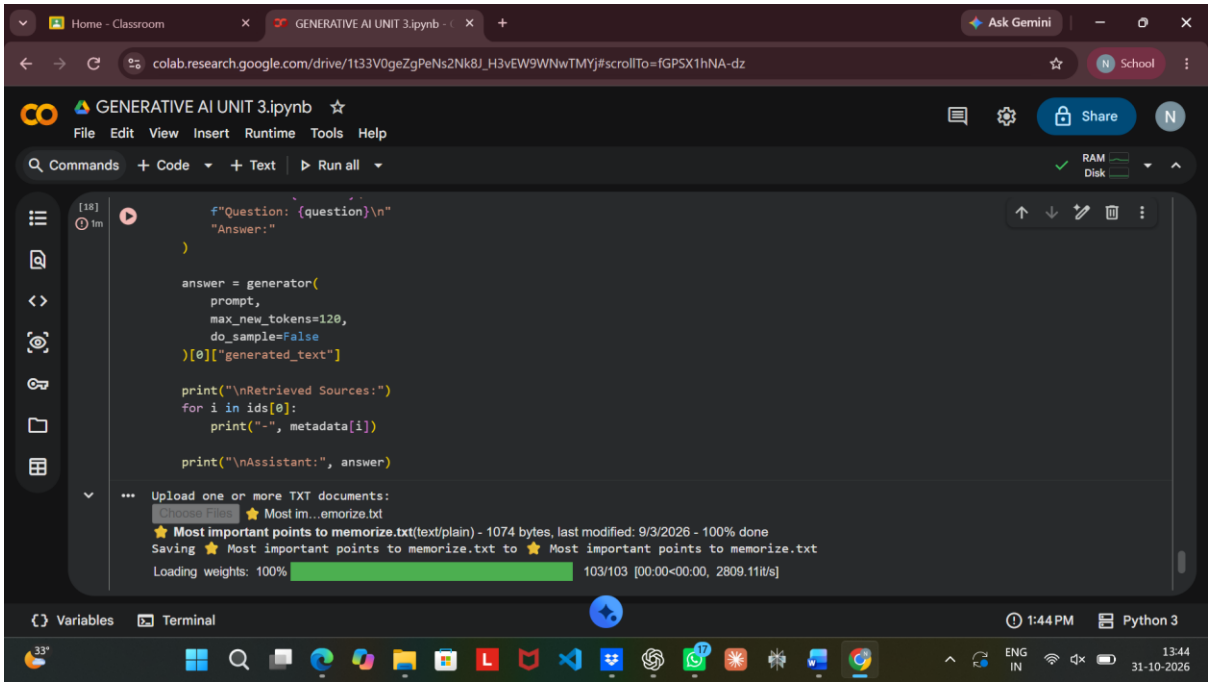
Retrieved Sources:

- ai.txt
- notes.txt

Assistant:

[Answer generated using the retrieved document content]

The file names and answer depend on the documents you upload.



RESULT

A document-based AI assistant was implemented to process multiple uploaded text documents, retrieve relevant passages and generate answers using RAG.